

Encrypt the cube, not the image.

DWT shrinks the images. A chaotic map encrypts the whole stack as one.

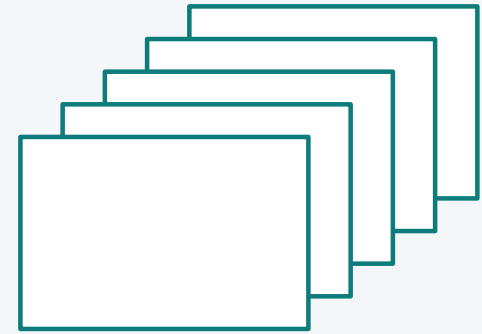
Multi-Image Encryption Scheme Based on Chaotic Pseudo-Random
Signal Generator and DWT Compression

ACM Transactions on Multimedia · Vol. 22, Issue 3 · 2026

DOI 10.1145/3769123 Yidan Xu · Suo Gao · Yinghong Cao · Jun Mou

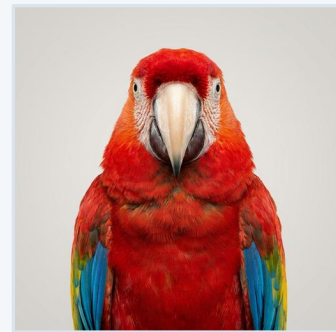
Abdul Rahman Malak (2285310)

Maria Alftaih (2285921)

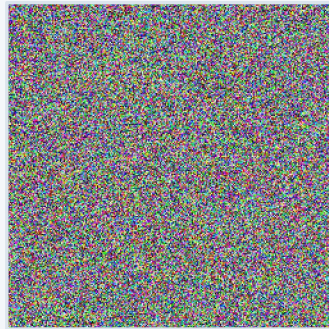
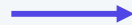


the cube

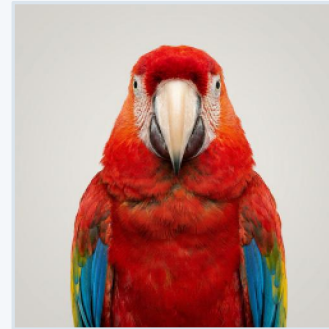
In plain English: what does this paper do?



a real image



encrypted = noise



decrypted back



one shared cube

- Send a whole batch of images that are smaller AND private at once.
- Shrink each image, then scramble them all together into noise.
- Only the secret key brings the images back.

About the paper.

TITLE

Multi-Image Encryption Scheme Based on Chaotic Pseudo-Random Signal Generator and DWT Compression

AUTHORS

Xu · Gao · Cao · Mou

VENUE

ACM Trans. on Multimedia

YEAR

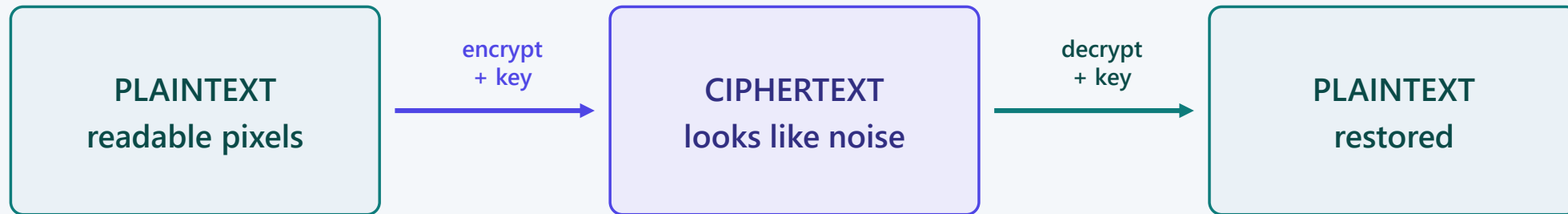
2026 · peer-reviewed

DOI

10.1145/3769123

- We chose it because the core idea is genuinely new.
- The maths stays approachable for a ten-minute talk.
- And the results are easy to show visually, as you'll see.

First, the basics: what is image encryption?



- Encryption turns readable pixels into noise using a secret key.
- Only the same key reverses it.
- Two moves: confusion shuffles positions, diffusion changes values.

Two goals, normally done separately.

SMALLER

Compression

shrink each image so a batch is cheap to send

SAFER

Encryption

scramble the pixels so the data is private

- Normally these are two separate, expensive steps.
- Most systems compress first, then encrypt — paying for both.
- This paper folds the two into a single pass.

What we'll cover in ten minutes.

01

Novelty

What is genuinely new?

Compress every image, stack them, encrypt the whole batch as one cube.

02

Methodology

How does it actually work?

DWT compression, a chaotic key stream, then confusion and diffusion.

03

Findings

Does it actually work?

32 dB recovered quality and five security tests, all in the ideal range.

- These are the three things the talk is graded on.
- Abdul takes the introduction and the novelty.
- Maria takes the methodology and the findings.

Sending image batches is heavy and exposed.

HEAVY

786,432 values per image

× N in a batch — scan series, satellite, surveillance.

EXPOSED

Raw pixels travel in the clear — anyone on the link can read them.

- The usual fix for size is compression.
- The usual fix for privacy is encryption.
- This paper does both — on the whole batch, in one pass.

Where does this actually matter?

Medical

MRI / CT scan series
shared between
hospitals

Satellite

large archives of earth-
observation imagery

Surveillance

banks of camera feeds
stored together

Cloud backup

a whole photo library
pushed up at once

- In every case you send many sensitive images together.
- Bandwidth is tight, so you want them small.
- They're private, so you want them encrypted.

Novelty

1

What is genuinely new in this paper?

Most schemes encrypt one image at a time. This one treats the whole batch as a single object.

An image is just a grid of numbers.



our test image

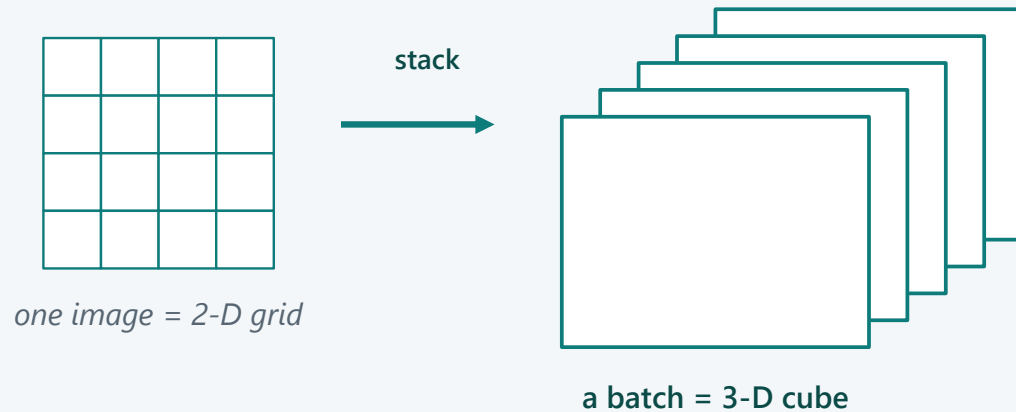


182	176	160	120	88
190	184	150	96	70
201	188	130	80	64
176	150	110	92	78
120	96	84	100	130

pixel values · 0–255

- A digital image is just a grid of numbers — one per pixel.
- Each value is a brightness from 0 to 255.
- So compressing or encrypting is really maths on a grid.

Stack the images into a 3-D cube.

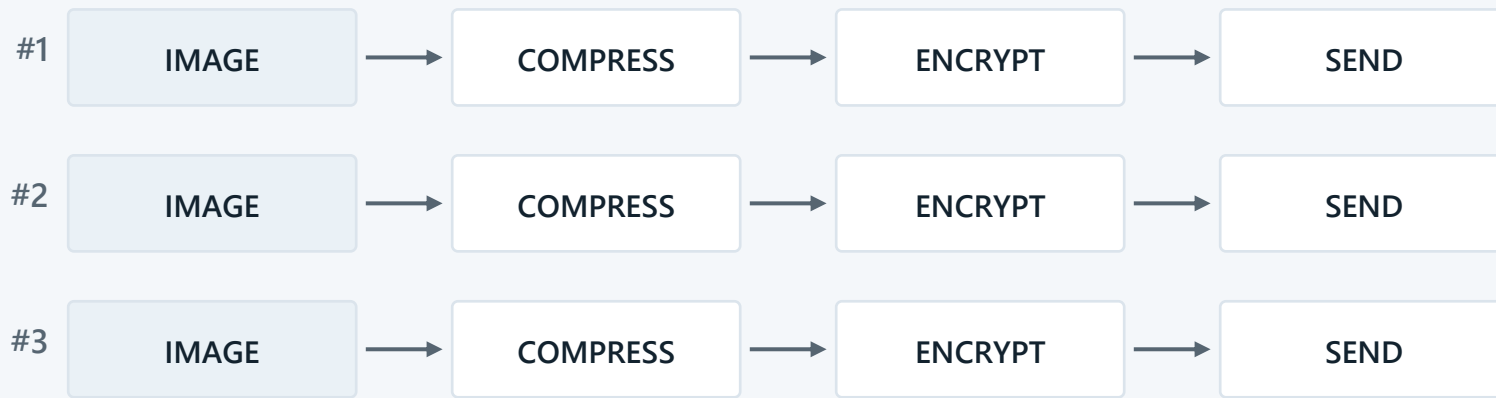


(i, j, k)

row · column ·
which image

- One image is a 2-D grid; a batch is a stack of grids.
- Stacked together they form one 3-D block — the cube.
- A pixel now needs three coordinates: (i, j, k) .

The old way: one image, one pipeline.



... repeat for every single image

COST PER BATCH

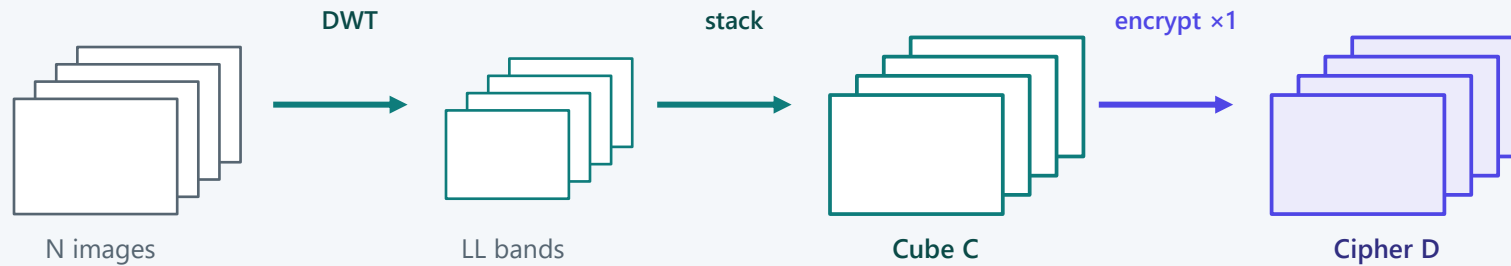
$2N$

processing steps

N keys

- Each image gets its own pipeline: compress, encrypt, send.
- N images $\rightarrow$ about $2N$ processing steps and N separate keys.
- It works, but it does not scale to large batches.

Encrypt the whole batch as one cube.

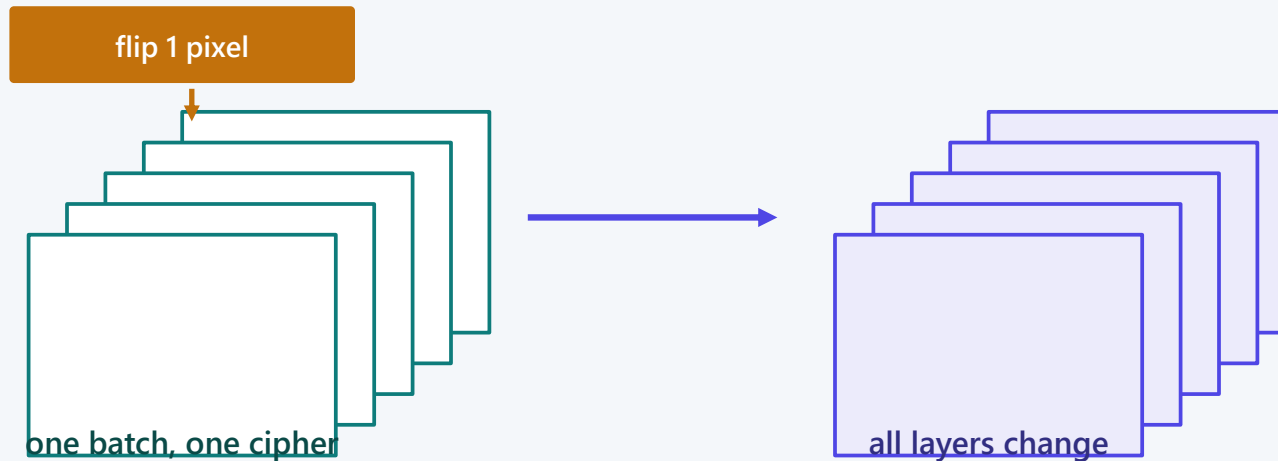


COST PER BATCH

~3 steps · 1 key

- Compress every image with DWT, keep only the LL part.
- Stack all the parts into one 3-D cube, C.
- Encrypt the whole cube once — about 3 steps, one key.

One cube means every image is entangled.



- All images share one cipher, so they are linked together.
- Flip one pixel in any image → every image's ciphertext changes.
- So an attacker can't isolate or attack a single image.

The recipe: two ingredients.

DWT

shrink

compress each image down to its essential part

Chaos

scramble

generate the key stream that hides the cube

- Together they are the whole method: shrink, then scramble.
- Both are driven by the secret key, so both reverse cleanly.
- Maria now shows exactly how each one works.

Methodology

2

How does the algorithm actually work?

Two ingredients — DWT and a chaotic map — combine through a four-stage pipeline.

Four stages, one key, one cube out.



- Stages 1–2 prepare the data; stages 3–4 are the encryption.
- The output is the cipher cube D.
- To decrypt, use the same key and reverse every stage.

Five words you'll hear throughout.

Plaintext

the original image data we want to protect

Ciphertext

the scrambled, noise-like output sent over the link

Key

the secret numbers that drive — and reverse — encryption

Confusion

rearranging where pixels sit (positions change)

Diffusion

changing what pixels are worth (values change)

- Keep these five straight and the rest of the talk follows.
- The two to watch: confusion (positions) versus diffusion (values).

One transform splits an image into four.



LL the rough visual base — half size — KEEP it

LH horizontal edge detail — drop

HL vertical edge detail — drop

HH diagonal edge detail — drop

- DWT is the wavelet cousin of the DCT used in JPEG.
- It separates the rough base from the fine edge detail.
- We keep the base because vision lives in low frequencies.

Keep LL. Drop three-quarters.



ORIGINAL · 512^2



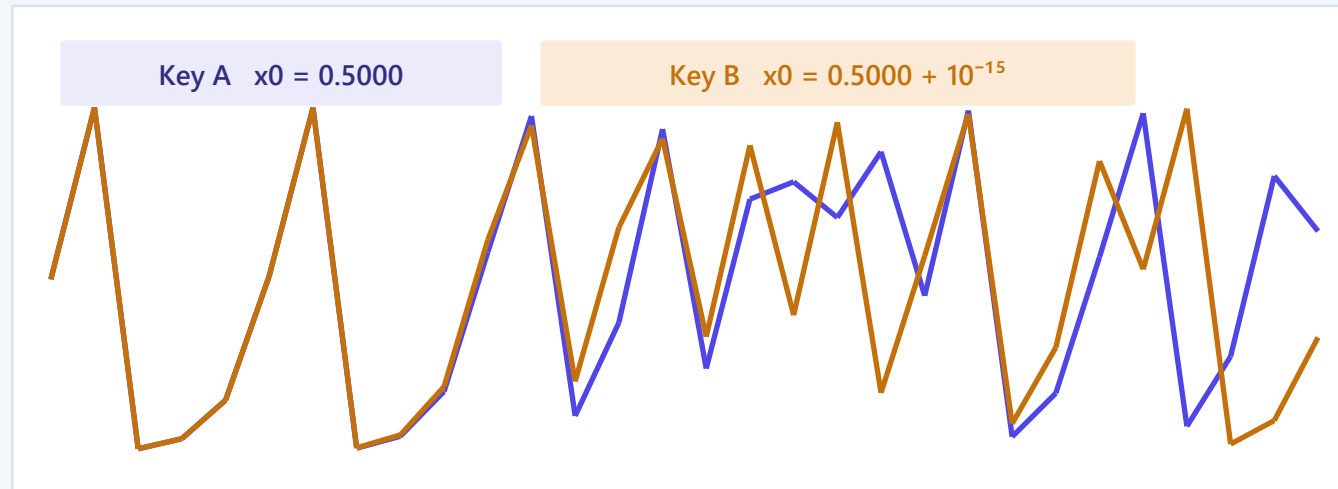
LL · 256^2

$\frac{1}{4}$ the area

single-level DWT · Eq. 4

- Keeping only LL drops 75% of the pixels.
- Vision lives in low frequencies, so the eye barely notices.
- Lossy, but the recovery still looks right.

A chaotic map is deterministic randomness.



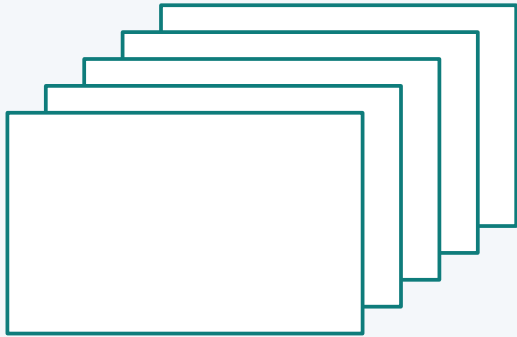
THE KEY = 9 NUMBERS

a b c d e f
x0 y0 z0

+ a 125,000-step burn-in,
tied to the image data

- The output looks random but is fully repeatable from its start.
- Change one bit of the key → a completely different stream.
- A 125,000-step burn-in makes it pass randomness tests.

Stack the LL bands into the plaintext cube C.



one layer per image channel

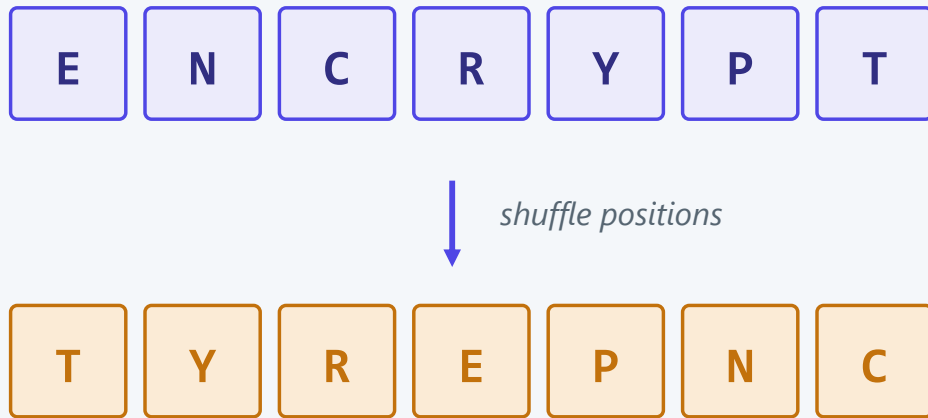
Cube C

$$(h_{\max}/2) \times (w_{\max}/2) \times \text{depth}$$

Eq. 5-6

- Each image's LL band becomes one layer of the cube.
- Smaller images are zero-padded to fit.
- Six numbers from the cube fold into the key, so it depends on the images.

Confusion: move positions, keep values.



VALUES

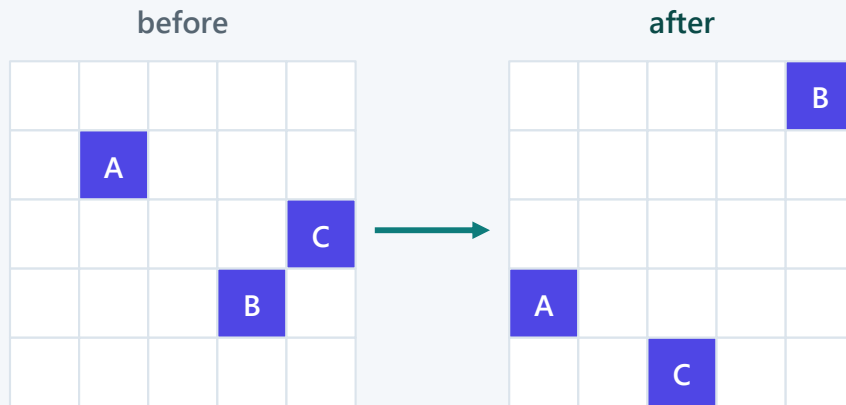
unchanged

POSITIONS

scrambled

- Confusion = move positions, keep the values.
- Like shuffling Scrabble tiles: same letters, nonsense board.
- Shannon's first requirement for a strong cipher.

How confusion chooses each move.

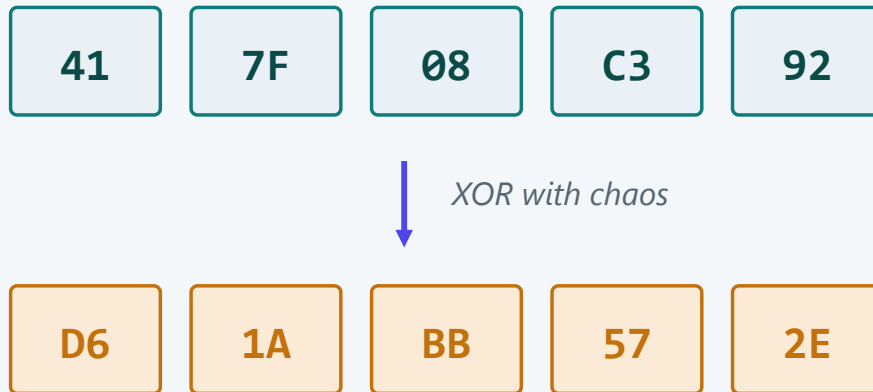


3 offsets → 9 swap cases

S1, T1, U1 · one per axis · Eq. 8–10

- The chaotic map decides every swap, so it's fully key-driven.
- Only the same key can walk the swaps backwards.
- Pixel values are never touched — this stage only moves them.

Diffusion: change values, keep positions.



POSITIONS

unchanged

VALUES

flipped (separate stream)

- Diffusion = change values, keep positions.
- Each value is XORed with a byte from a separate chaos stream.
- Separate stream — reusing confusion's would leak structure.

The diffusion equation.

$$D[k] = V[k] \oplus \text{chaos}[k \bmod 3] \oplus D[k-1]$$

$V[k]$

value from confusion

$\text{chaos}[k \bmod 3]$

a byte from the chaos stream

$D[k-1]$

previous cipher byte = the chain

- Cipher byte = confused value XOR chaos byte XOR the previous byte.
- That last term chains outputs — like CBC mode at the pixel level.
- XOR is its own inverse, so the same key undoes it.

One flipped bit cascades through everything.



- Each byte depends on the one before it.
- Flip one bit at the start → every later byte changes.
- That cascade is why NPCR $\approx 99.65\%$.

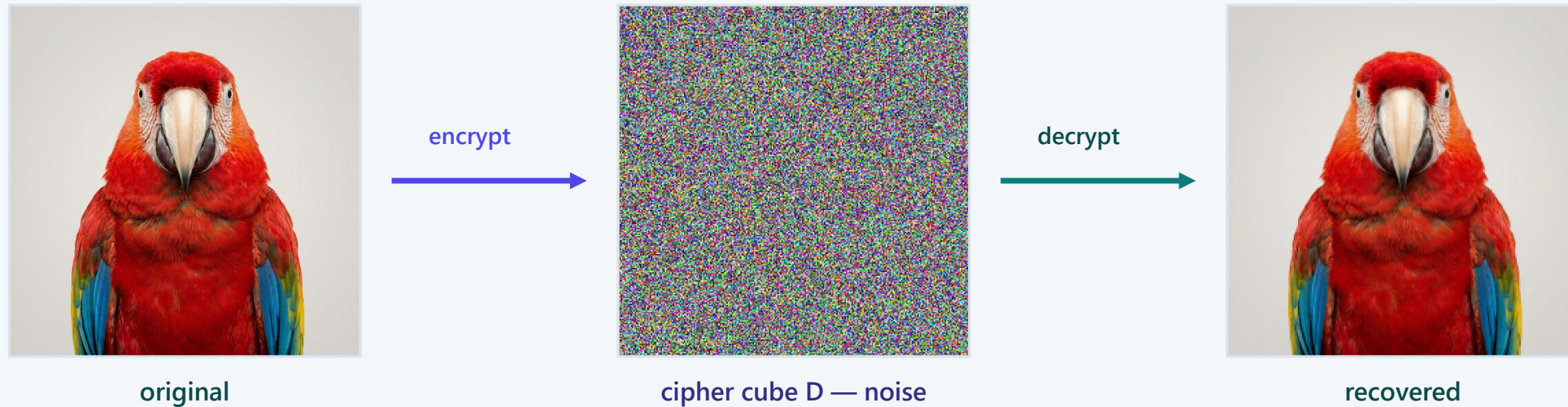
Findings

Does it actually work?

Near-lossless recovery, and five standard security tests that all land in the ideal range.



Encrypt to noise, decrypt back.



- Encrypt the cube → it becomes pure noise.
- Decrypt with the key and inverse-DWT → the image returns.
- Our Haar demo recovers at 32.16 dB $\approx$ the paper's 32.06 dB.

Recovered quality: 32.06 dB.

Average PSNR over 12 test images

32.06 dB

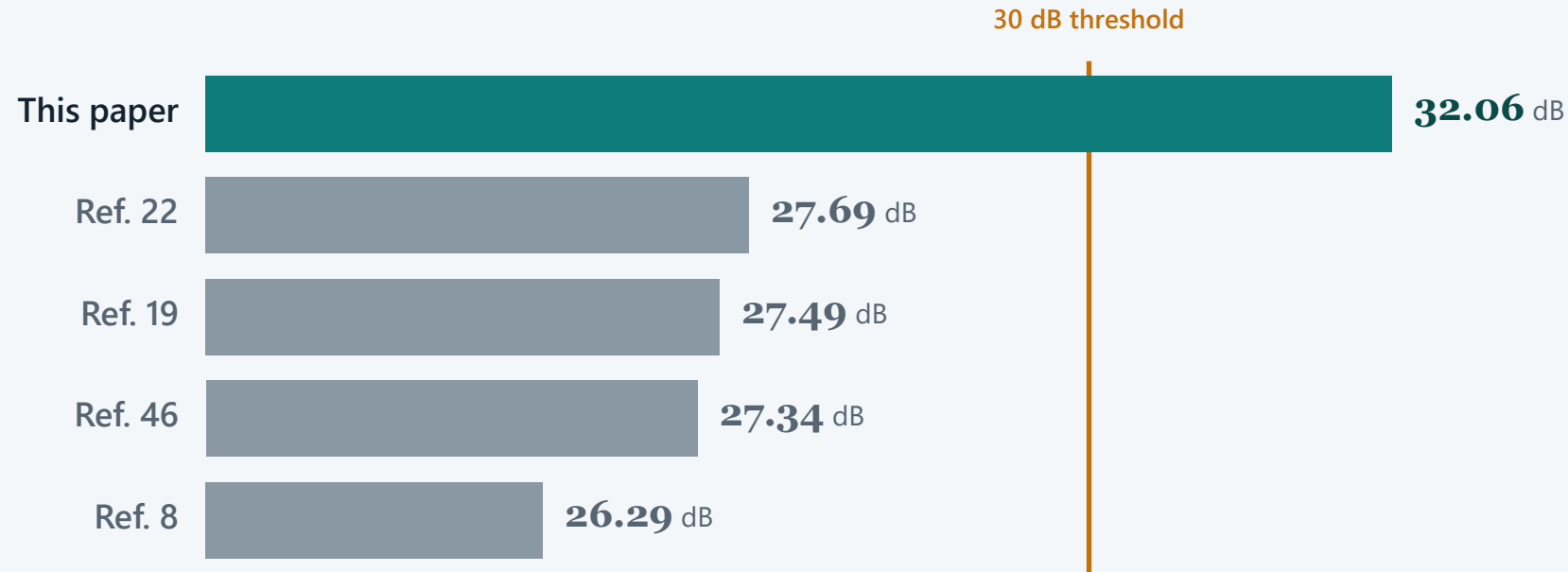
RULE OF THUMB

above 30 dB

= visually identical to the original.

- Average PSNR is 32.06 dB over 12 images.
- That's after compression AND a full encrypt-decrypt cycle.
- Above 30 dB = visually identical to the original.

Above the line; prior work below it.



- Prior chaos-based schemes sit at 26–28 dB; this one reaches 32.06.
- PSNR is logarithmic — a 4–6 dB gap $\approx$ 2.5–4 $\times$ less error.

The cipher behaves like pure noise.



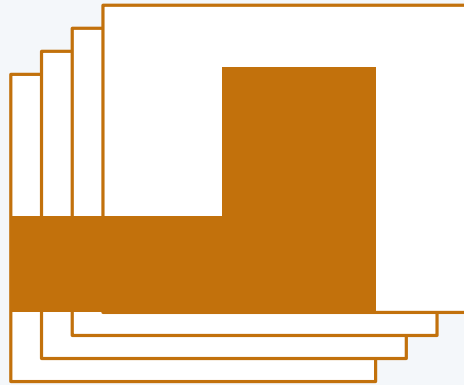
- Histogram goes spiky → flat, so frequency analysis fails.
- Adjacent-pixel correlation collapses to a random cloud.

Five standard tests — all in the ideal zone.

	ACHIEVED	TARGET
NIST suite	15 / 15 pass	$p \geq 0.01$
NPCR	99.6533%	99.6094%
UACI	33.4887%	33.4635%
Entropy	7.9994 / 8	8.0000
Key space	$\approx 2^{448}$	$> 2^{100}$

- Every test lands at or beyond its ideal target.
- Honest caveat: strong statistical evidence — not a formal proof like AES.

The cost: zero-padding and a slow chain.



wasted padding

Two honest costs

- Different sizes → zero-padding that's still encrypted and sent.
- ~6 MB/s vs ~500 for AES — the diffusion chain is serial.
- Authors flag the padding as future work.

One pipeline: shrink, stack, scramble.

DWT

handles size

Chaos

handles secrecy

Cube

handles batches

Compression and encryption · multiple images · one pass.

Questions?

Thank you.